

Gradescope: To submit this homework assignment, use Gradescope accessed through Canvas. The assignment could be either typed using LaTeX or handwritten and scanned.

Problem 1 (Maximum Likelihood Estimation, 10 points) Consider a random variable $\mathbf{X}$ (possibly a vector) with probability density function written as $f(\mathbf{x}; \boldsymbol{\theta})$, where $\boldsymbol{\theta}$ is called the parameter, and can be either a scalar or vector. Maximum likelihood estimation is one of the most important parameter estimation techniques. Let $\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_n$ be i.i.d (independent and identically distributed) random variables with the joint distribution as the product $\prod_{i=1}^n f(\mathbf{x}_i; \boldsymbol{\theta})$. Viewed as a function of $\boldsymbol{\theta}$, this quantity is called the *likelihood* of $\boldsymbol{\theta}$.

- (a) (5 points) You are independently drawing n samples $x_1, x_2, \dots, x_n$ from a uniform distribution $U(a, b)$. Please show the likelihood of $\boldsymbol{\theta} = \{a, b\}$, and estimate $\boldsymbol{\theta}$ from the observed data.
- (b) (5 points) You have a bunch of light bulbs, whose lifetime can be modeled by an exponential distribution with unknown parameter λ . After testing n bulbs, you observe that the lifetime of these n bulbs are $x_1, x_2, \dots, x_n$, respectively. Please show the likelihood of $\boldsymbol{\theta} = \{\lambda\}$, and estimate $\boldsymbol{\theta}$ from the observed data.

Hint: The PDF of an exponential distribution with parameter λ is:

$$f_X(x) = \begin{cases} \lambda e^{-\lambda x}, & x \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

Problem 2 (The Bayes Classifier, 20 points total) Consider a binary classification problem involving a single (scalar) feature x and suppose that $X|Y = 0$ and $X|Y = 1$ are continuous random variables with densities given by

$$f_{X|Y}(x|0) = \begin{cases} e^{-x}, & x \geq 0 \\ 0, & \text{otherwise,} \end{cases}$$
$$f_{X|Y}(x|1) = \begin{cases} 1/2, & x \in [a, a+2], \\ 0, & \text{otherwise,} \end{cases}$$

respectively, where $a \geq 0$ is given. Furthermore, suppose that $\mathbb{P}[Y = 0] = \mathbb{P}[Y = 1] = \frac{1}{2}$.

- (a) (4 points) Sketch $f_{X|Y}(x|0)$ and $f_{X|Y}(x|1)$.
- (b) (6 points) Suppose $X = 1/2$; what is the most likely value for Y according to the Bayes rule? What about $X = 1$? (Hint: the solution may depend on the value a)
- (c) (10 points) State the optimal classification rule (i.e., the Bayes classifier) for any possible $x \geq 0$. (Hint: the solution depends on the value a)

Problem 3 (Linear Discriminant Analysis, 15 points) Suppose we are given a dataset $\{(\mathbf{x}^{(i)}, y^{(i)}); i = 1, 2, \dots, N\}$ consisting of N independent examples, where $\mathbf{x}^{(i)} \in \mathbb{R}^M$ are M -dimensional vectors, and $y^{(i)} \in \{0, 1\}$. We will model the distribution of $(\mathbf{x}^{(i)}, y^{(i)})$ using:

$$p(y^{(i)}) = \phi^{y^{(i)}} (1 - \phi)^{1-y^{(i)}},$$

$$p(\mathbf{x}^{(i)} | y^{(i)} = 0) = \frac{1}{(2\pi)^{\frac{M}{2}} |\Sigma|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(\mathbf{x}^{(i)} - \mu_0)^T \Sigma^{-1} (\mathbf{x}^{(i)} - \mu_0)\right)$$

$$p(\mathbf{x}^{(i)} | y^{(i)} = 1) = \frac{1}{(2\pi)^{\frac{M}{2}} |\Sigma|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(\mathbf{x}^{(i)} - \mu_1)^T \Sigma^{-1} (\mathbf{x}^{(i)} - \mu_1)\right)$$

Here, Σ is known. The parameters of our model (the LDA classifier) are ϕ , μ_0 , and μ_1 .

(a) (5 points) Please write the maximum likelihood estimation of the parameters of our model.

Hint 1: You can directly write out the solution.

Hint 2: You may find the [Indicator function](#) useful.

(b) (10 points) Suppose we have already fit ϕ , μ_0 , and μ_1 , and now want to make a prediction at some new query point $\mathbf{x}$. Show that the posterior distribution of the label y at $\mathbf{x}$ takes the form of a logistic function, and can be written as:

$$p(y = 1 | \mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x} + b)}.$$

Please give the expression of $\mathbf{w}$ and b , w.r.t. ϕ , Σ , μ_0 , and μ_1 . Please also briefly describe how to classify the new query point $\mathbf{x}$ accordingly.

Hint 1: Use Bayes rule.

Problem 4 (Linear Regression, 30 points) In this problem we will compare the performance of traditional least squares, ridge regression, and the LASSO on a real-world dataset. We will use the Boston House Prices dataset which contains the median sale price (as of some point in the 1970's, when the dataset was created) of owner occupied homes in about 500 different neighborhoods in the Boston area, along with 13 features for each home that might be relevant. These features include factors such as measures of the crime rate; measures of school quality; various measures of density; proximity to things like highways, major employment centers, the Charles River; pollution levels; etc.¹ To get this dataset in Python, simply type

```
import numpy as np
import pandas as pd
# load_boston was removed from scikit-learn 1.2;
# this loads the same data from its original source
data_url = "http://lib.stat.cmu.edu/datasets/boston"
raw = pd.read_csv(data_url, sep=r"\s+",
                  skiprows=22, header=None)
X = np.hstack([raw.values[:,2, :], raw.values[1::2, :2]])
y = raw.values[1::2, 2]
```

In order to judge the quality of each approach, you should split the dataset into a training set and a testing set. The training set should consist of the first 400 observations (**Note:** For grading purpose, please do not use sklearn's `train_test_split`), and you can use the remaining observations for testing.

```
X_train = X[0:400]
y_train = y[0:400]
X_test = X[400:]
y_test = y[400:]
```

Before training any of these algorithms, it is a good idea to “standardize” the data. By this, I mean that take each feature (i.e., each column of the matrix X) and subtract off its mean and divide by the standard deviation to make it zero mean and unit variance. Otherwise, the regularized methods will implicitly be placing bigger penalties on using features which just happen to be scaled to have small variance. This can be done by using

```
from sklearn import preprocessing
X = preprocessing.scale(X)
```

For all parts of the problem below, no starting code will be provided. **Please attach your code in your Homework. You will lose all points without attaching your code.**

- (a) (10 points) First, I would like you to evaluate the performance of least squares. You should implement this yourself using the equation derived in class. Report the performance of your algorithm in terms of mean-squared error (MSE) on the test set, i.e.,

$$\frac{1}{n_{\text{test}}} \| \mathbf{y}_{\text{test}} - \mathbf{A}_{\text{test}} \hat{\boldsymbol{\theta}} \|^2_2$$

¹See <http://bit.ly/2lTueYY> for more details about this dataset.

where $\mathbf{A}_{\text{test}} = [\mathbf{1} \quad \mathbf{X}_{\text{test}}]$ adds a feature with value 1 to each data point. This is a notational shortcut that saves us from having to write the bias terms explicitly in our code and analysis.

- (b) (10 points) Next, using the formula derived in class (see the lecture slides on ridge regression), implement your own version of ridge regression. You will need to set the free parameter λ . You should do this using the training data in whatever manner you like (e.g., via a validation set)—but you should not allow the testing dataset to influence your choice of λ . Report the value of λ selected and the performance of your algorithm in terms of mean-squared error on the test set.
- (c) (10 points) Finally, I would like you to evaluate the performance of the LASSO. You do not need to implement this yourself. Instead, you can use scikit-learn's built in solver via:

```
from sklearn import linear_model
# instantiate and train a Lasso model
reg = linear_model.Lasso(alpha = ???)
reg.fit(Xtrain,ytrain)
# use the trained model to predict ytest from Xtest
ytest = reg.predict(Xtest)
```

Above, alpha corresponds to the λ parameter from the lecture notes. As in part (b), you will need to do something principled to choose a good value for this parameter. Report the value of alpha used in your code, the performance of your algorithm in terms of mean-squared error, and the number of nonzeros in $\boldsymbol{\theta}$. (You may find this [documentation](#) helpful)

Note: the input data to `reg.fit` and `reg.predict` is $\mathbf{X}$ instead of $\mathbf{A}$ since the LASSO algorithm in scikit-learn will augment the data matrix with column $\mathbf{1}$ by itself.

Problem 5 (Logistic Regression, 45 points) In this problem you will explore implementations of two different solvers for logistic regression. To get started, download the file `lr-gd.py` from canvas. **Please attach your code in your Homework. You will lose all points without attaching your code.**

The *log-likelihood* of the logistic regression model is given by:

$$\ell(\theta) = \sum_{i=1}^n y_i \log(g(\theta^T \tilde{x}_i)) + (1 - y_i) \log(1 - g(\theta^T \tilde{x}_i)),$$

where $g(t) \triangleq \frac{1}{1 + \exp(-t)}$ is the logistic function. For all parts below, keep in mind our goal is to *minimize* the *negative log-likelihood*, i.e., minimize $-\ell(\theta)$.

- (a) (10 points) We will begin by implementing logistic regression using standard gradient descent for performing the maximum likelihood estimation step. Review the lecture slides on logistic regression and gradient descent and make sure you understand what `lr-gd.py` is doing. Test out the file by experimenting a bit with the various parameters—especially the “step size” or learning rate α —to obtain a good convergence rate. I would recommend trying a wide variety starting with $\alpha \approx 1$ and ranging to $\alpha \ll 1$. For this problem, all you need to do is to report the value(s) of α you used, the number(s) of iterations required for convergence and the final negative log-likelihood (which is the last element of `cost`) for your choices of α . Please also paste your code of the function `grad_desc()`. You should not change the default `tol` and `maxiter` for your report.

Hint: You should report at least 3 different “step size” you have tried, as well as the corresponding iterations and the final negative log-likelihood.

- (b) (10 points) Show that the Hessian of the *log-likelihood* can be written as:

$$\frac{\nabla^2 \ell(\theta)}{\partial \theta^2} = - \sum_{i=1}^n \tilde{x}_i \tilde{x}_i^T g(\theta^T \tilde{x}_i) (1 - g(\theta^T \tilde{x}_i)),$$

where g is defined as above.

- (c) (15 points) Using the lecture notes on Newton’s method, adapt `lr-gd.py` to implement Newton’s method for solving this problem. This algorithm requires computing both the gradient and the Hessian at each iteration.

Take extra care in your implementation in making sure you have the correct signs in your algorithm—remember we are trying to *minimize* the *negative log-likelihood*. If your algorithm is not converging, this is a likely suspect. Note also that in the notes I assume that the $\tilde{x}$ are column vectors, but in the Python code they are treated as row vectors.

Just as a reminder, in Python `A*B` computes the element-wise multiplication, while `A.dot(B)` computes matrix multiplication. You may also want to compute a matrix inverse of A using `np.linalg.inv(A)`. You should complete this assignment by filling the code block of `newton_method()`. You may find it helpful to design other helper functions (such as `log_hessian()`, a function that computes the Hessian of the log-likelihood). Submit the code you produce for this function (do not include the plotting commands), and report the number of iterations required for convergence and the final negative log-likelihood (which is the last element of `cost`). Compare this to the results from part (a).

- (d) (10 points) Finally, try comparing the results from parts (a) and (c) when applied to a much larger dataset. Try changing `n_samples` to 100000. You will want to comment out the plotting portion

of the code for this part (or at least the part that plots the training data). Record the final loss, running time and the number of iterations for both algorithms when applied to this dataset. Note that in order to run traditional gradient descent on this large of a dataset, you will need to make α much smaller (think about why this would be the case). If you see warnings regarding a divide by zero or the cost is much larger than the one obtained by Newton's method (or even `inf`), you have set α to be too large. Also, note that you can get the system time in Python by adding `import time` to your file and then using the `time.time()` command.